



Inspiring Excellence

CSE370 : Database Systems
Project Report
Project Title : Gamezone Management System

Group No : 03, CSE370 Lab Section : 11, Spring 2026		
ID	Name	Contribution
23201404	Aniruddha Saha	Access Management System, Tournament Management and Player Dashboard
23201155	Mazharul Islam Sadid	Room Management, Analytics Dashboard, Payment
23201563	Rahik Al Wasif	Room Availability, Membership System, Regular Game Management

Table of Contents

Section No	Content	Page No
1	Introduction	03
2	Project Features	04
3	ER/EER Diagram	05
4	Schema Diagram	06
5	Normalization	07-08
6	Frontend Development	09-11
7	Backend Development	12-14
8	Source Code Repository	15
9	Conclusion	15
10	References	15

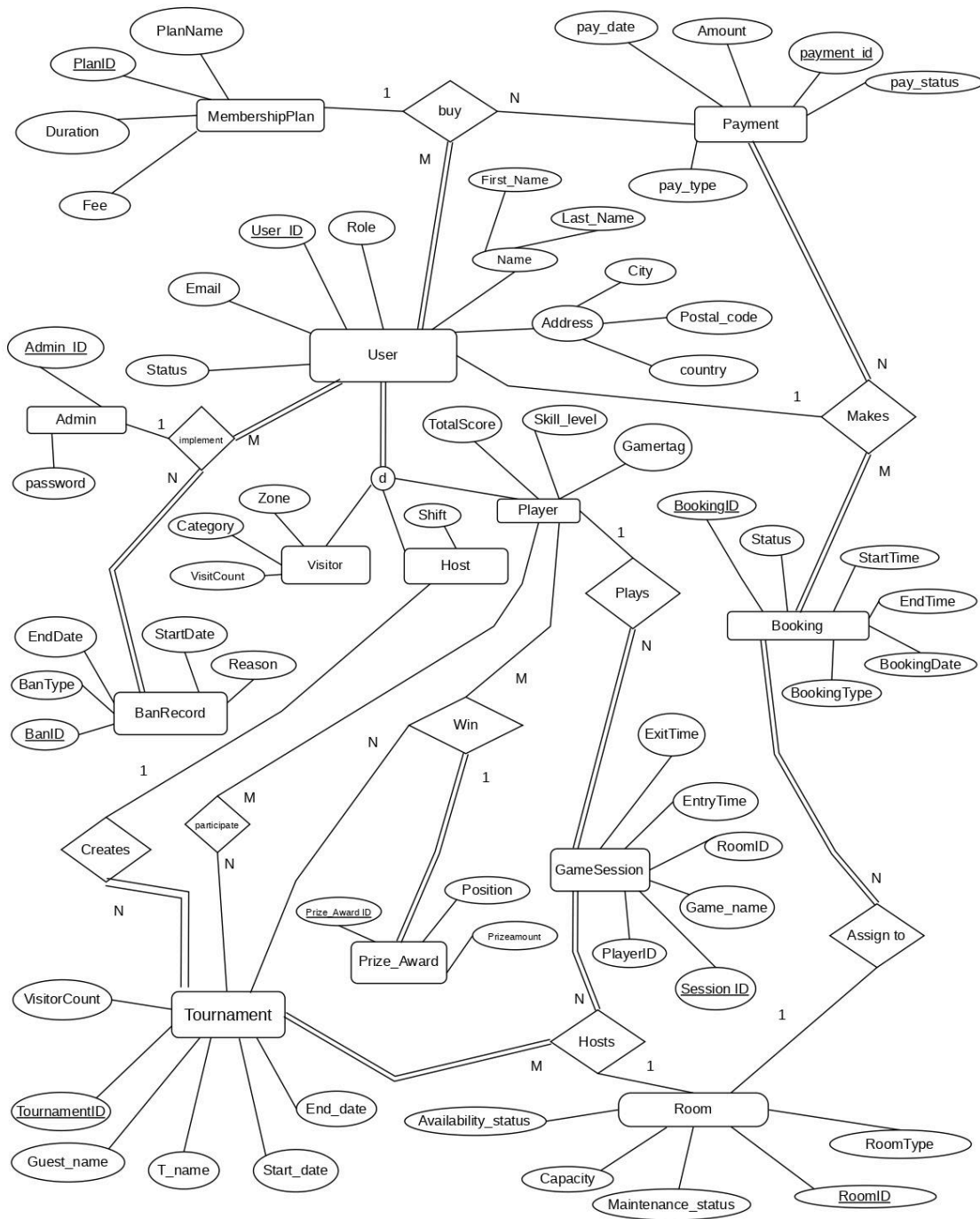
Introduction

We took inspiration from E-Sports tournaments for our project named GameZone. It is an online platform where games can be played and E-Sports tournaments can be hosted. There is an Admin who acts as the owner and maintains the website. People can sign up as a host, visitor and player. The host can host the tournament, assign prize money and award the winning players. The players can book rooms for casual gaming or join any available tournament. The visitor can join tournaments as spectators only. All the users except Admin have separate membership plans. These roles combinedly create an engaging environment where competitive gaming, casual play, and live spectatorship all come together in one area.

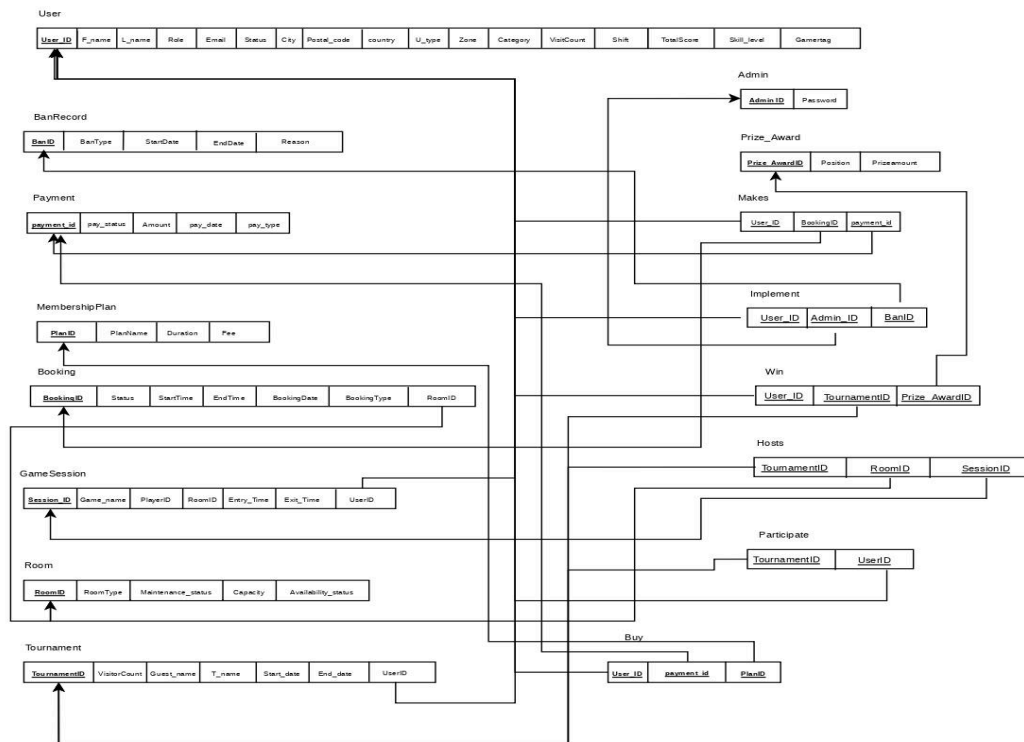
Project Features

ID, Name	Features [3 per member]	
23201404, Aniruddha Saha	Ft 1	Access Management System [role-based access and Ban/Suspension records] , Assigning games for created regular room by Admin, Player Part [Tournament join ,Player dashboard]
	Ft 2	Tournament Management [Tournament creation, participant registration, leaderboard, and visitor count,Capacity control]
	Ft 3	
23201155, Mazharul Islam Sadid	Ft 1	Room Management [Regular and Tournament zone booking, capacity tracking]
	Ft 2	Analytics Dashboard [Separate dashboards for Admin, Host, Visitor. Showing analytic breakdown of each user, Payment {(room booking payments, refund tracking, and tournament prize pool distribution)]
	Ft 3	
23201563, Rahik Al Wasif	Ft 1	Room Management [availability status, and maintenance logs, Membership System (Membership plans, separate membership plan for each)
	Ft 2	Regular Game Management [Player room booking, clash managing , and visitor count]
	Ft 3	

ER/EER Diagram



Schema Diagram

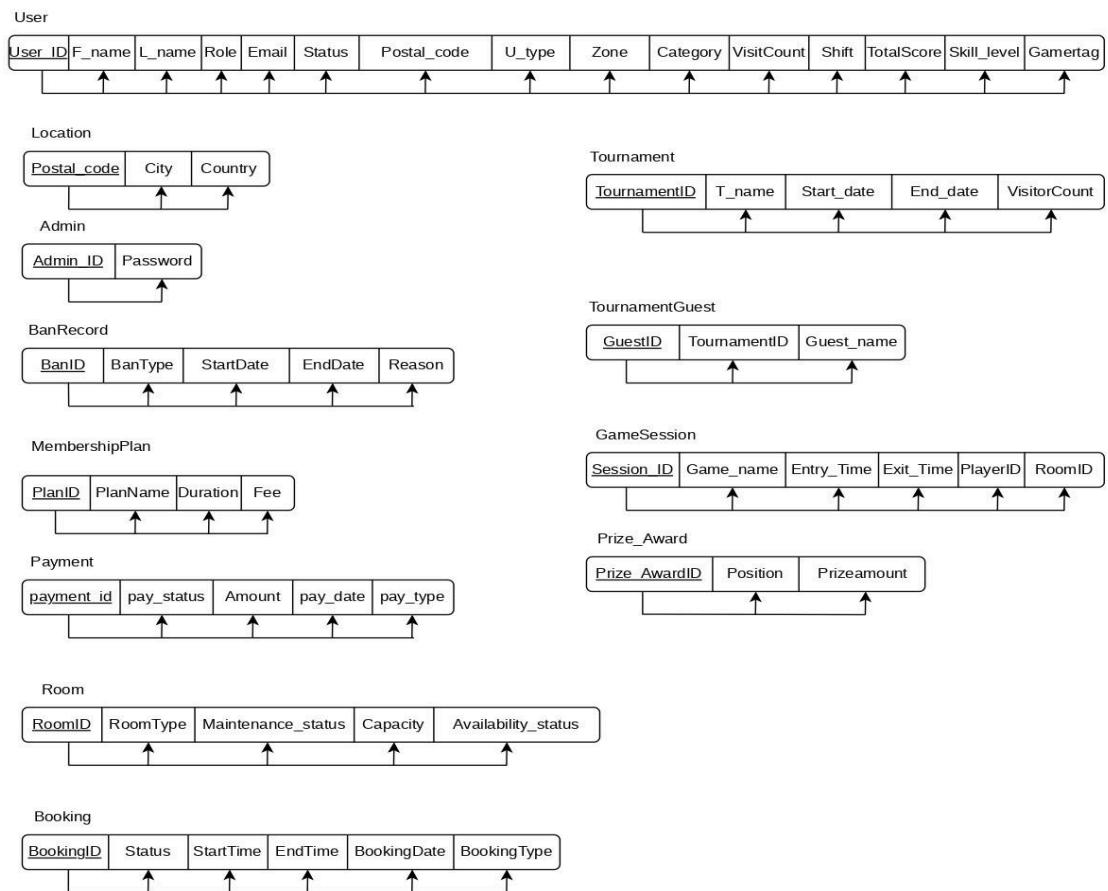


Normalization

- a. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.
- b. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?
- c. Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?

ANS:

- a. Our converted schema is in 1NF as there are no multivalued or composite attributes and no nested relations. For example, the User table stores Postal_code, country and City as separate atomic attributes rather than a composite Address.
- b. Our converted schema is in 2NF as there cannot be any partial dependencies in the relational schema. Here, all non-key attributes are fully dependent on the entire primary key. Partial dependencies happen when a table has a composite primary key and a non-key attribute depends on only part of that key.
- c. Our converted schema is not in 3NF. Postal Code has transitive dependency on City and Country. Transitive dependencies cannot be in 3NF. Transitive dependencies happen when a non-key attribute defines another non-key attribute. In our schema, the only transitive dependency was Postal_code $\rightarrow$ City $\rightarrow$ Country in the User table. We separated that from the user table in the 3NF form. The 3NF form is:



Frontend Development

Contribution of ID : 23201404 Name : Aniruddha Saha

- The access management system is implemented across multiple PHP files supporting role-based login/authentication. The login.php validates user credentials and redirects users to their appropriate dashboards (Admin, Host, Player, Visitor) based on their role. Access control checks are enforced on each dashboard using session variables, and banned/suspended users receive appropriate error messages. The admin/dashboard.php includes ban management features where admins can view suspension/ban records, assign games to created regular rooms via game assignment forms, and manage user status.
- The player dashboard (player/player.html) provides tournament joining functionality with player/handle_tournament.php handling tournament registration. The dashboard displays available tournaments with entry fees, player limits, status badges, and join buttons. Players can view their registered tournaments, check participation status, and access their membership information through player/dashboard.php.
- Tournament management for admin is implemented in the admin/ directory with comprehensive control over all tournaments. The admin/create_tournament.php builds tournament creation forms with fields for tournament name, game selection, start/end dates, entry fee, prize money, player limits, and visitor limits. The admin/tournament_list.php displays all tournaments with participant registration details, visitor counts, capacity indicators showing filled vs. maximum slots, status badges (registration_open, in_progress, completed, cancelled), and action buttons for edit, cancel, or view participants. The leaderboard display shows participant rankings with scores, positions, and prize distributions accessible via tournament details. Capacity control is enforced by displaying current visitor/player counts against limits and disabling further registrations when tournaments reach capacity.

Contribution of ID : 23201155, Name : Mazharul Islam Sadid

- Room management is implemented across admin/, host/, player/, and visitor/ directories. The admin/room_create.php builds room creation forms with room name, room type (regular/tournament), capacity input, and available games dropdown. The admin/room_list.php displays all rooms in tables with edit/delete actions and status badges. Host room booking forms in host/room_booking.php include date/time pickers, game selection, player count, and visual capacity tracking showing used slots against room capacity.
- Visitor booking in visitor/booking.php displays available rooms with availability status, capacity indicators, game tags, and booking confirmation buttons. Tournament zone bookings follow similar patterns with visitor_limit displays showing filled vs. maximum slots and disabled buttons when full.
- Payment and refund tracking displays across player/dashboard.php and visitor/dashboard.php showing membership refunds (40% calculation), tournament entry refunds with status badges, and transaction history with dates and amounts. Tournament prize pool distribution is displayed in tournament details showing prize_money, participant counts, winner badges (1st, 2nd, 3rd), and prize distribution status.
- Analytics dashboards provide role-specific insights. Admin dashboard shows users by role, total revenue, active tournaments, room occupancy rates with progress bars, and transaction logs. Host dashboard displays tournaments hosted, visitor counts, revenue from visitor entries (60% share), and participation stats. Visitor analytics shows visit history, tournament participation, membership status/expiry, total spending, and personal performance rankings. All use Bootstrap cards, badges, and tables for data visualization.

Contribution of ID : 23201563, Name : Rahik Al Wasif

- Room availability status and maintenance logs are shown in admin/room_list.php and host/dashboard.php. Status badges display availability (available/occupied/reserved) and maintenance status (good/maintenance_needed/under_maintenance). Maintenance history logs show dates and reasons with color-coded indicators.
- Membership system in admin/create_membership.php builds plan creation forms with plan_name, duration, fee, description, and role selection (visitor/player/host) for separate plans per role. Visitor/player/host see only their applicable plans with purchase buttons showing duration, fee, and expiry calculations.
- Player room booking in player/room_booking.php includes room selection, date/time pickers, game dropdown, and player count input with capacity tracking. Clash warnings appear when selected time conflicts with existing bookings. The player/booking_list.php shows bookings with status badges and cancel options.
- Visitor count displays in host/dashboard.php showing total visitors, active per room, and historical counts with tables and progress bars for capacity utilization.

Backend Development

Contribution of ID : 23201404, Name : Aniruddha Saha

- The access management system is implemented in login.php which validates user credentials against the User table using password_verify(). The selected role from the login form is compared with the user's role stored in the database. If matched and status is active, the user is redirected to their role-specific dashboard (admin/dashboard.php, host/dashboard.php, player/dashboard.php, or visitor/dashboard.php). Session variables store user_id, username, email, role, and status for access control throughout the application.
- The admin/ban_user.php requires Admin role and handles both suspension and permanent ban operations. It reads user_id and ban_type from POST, generates ban_id using time-based logic, and inserts into BANRECORD with start_date, optional end_date for suspensions, and reason text. The admin/unban_user.php removes ban records and updates user status to active, allowing previously suspended or banned users to login again.
- The player/handle_tournament.php requires a Player role and processes tournament join requests. It validates tournament_id, checks if the tournament status is registration_open, verifies player_limit has not been reached, and checks if the player is not already registered in PARTICIPATE. It generates participant_id and inserts into PARTICIPATE with status registered.
- The player/dashboard.php loads the player's registered tournaments by querying PARTICIPATE joined with TOURNAMENT, displays registration dates, tournament names, entry fees, and status badges. It also loads active membership information from BUY joined with MEMBERSHIPPLAN where the end_date is greater than or equal to current date.
- Tournament management for admin is handled in admin/create_tournament.php which requires Admin role. Upon form submission, it validates t_name, game_name, start_date, end_date, entry_fee, prize_money, player_limit, and visitor_limit. It generates tournament_id using time-based logic and inserts into TOURNAMENT with status registration_open, current visitor_count as 0, and max_participants based on player_limit.
- The admin/tournament_list.php queries all tournaments from TOURNAMENT with host user details, displays tournament status with color-coded badges, and calculates filled capacity by joining with PARTICIPATE for player_count and VISITOR_TOURNAMENT for visitor_count. Capacity control is enforced by comparing current counts against player_limit and visitor_limit before allowing registrations.
- The admin/tournament_details.php loads individual tournament information by tournament_id, displays participant lists from PARTICIPATE with user scores, and shows visitor registrations from VISITOR_TOURNAMENT. The leaderboard is generated by querying PARTICIPATE joined with USER ordered by total_score descending, displaying position, player name, score, and prize information.

Contribution of ID : 23201155, Name : Mazharul Islam Sadid

- Room management in admin/room_create.php validates room_name, room_type, capacity, and available_games, generates room_id, and inserts into ROOM with availability_status available and maintenance_status good. The room_list.php loads rooms with optional status filters from GET parameters.
- Host room booking in host/room_booking.php validates booking_date, start_time, end_time, game_name, and num_players, checks capacity and time conflicts by querying overlapping BOOKING records, generates booking_id, and inserts with status confirmed and booking_type regular. Visitor booking follows similar validation with membership check from BUY table.
- Payment processing in payment/booking_payment.php calculates fees based on room type/duration, inserts PAYMENT with pay_type booking and pay_status pending. Refund logic calculates 40% of original fees from original PAYMENT records, inserts refund PAYMENT with pay_type refund, and updates original payment status to refunded.
- Prize pool distribution in admin/tournament_details.php queries prize_money, calculates prize tiers, and inserts WIN records linking winners to prize_award. Host revenue is calculated by querying VISITOR_TOURNAMENT entries: positive user_id at full fee plus negative user_id at 60% fee.
- Analytics backend aggregates data from multiple tables. Admin: users by role from USER, revenue from PAYMENT where pay_status completed, active tournaments from TOURNAMENT, room occupancy from BOOKING/ROOM. Host: tournaments hosted, visitor counts, revenue from VISITOR_TOURNAMENT. Visitor: visit_count, tournament history from PARTICIPATE/VISITOR_TOURNAMENT, spending from PAYMENT joined with Makes.

Contribution of ID : 23201563, Name : Rahik Al Wasif

- Room availability status management is implemented in host/booking.php and admin/edit_room.php. When a booking is confirmed, it updates the room's availability_status to occupied via: UPDATE ROOM SET availability_status='occupied' WHERE room_id=?; When booking is completed or cancelled, it updates back to available. The status check queries ROOM table by room_id before allowing new bookings.
- Maintenance logging in admin/maintenance_log.php records status changes. When admin changes maintenance_status to maintenance_needed or under_maintenance via edit_room.php, it logs the change by storing status, reason text, created_at timestamp, and admin_id. The maintenance_history.php queries BANRECORD table ordered by created_at descending for display.
- Membership system in admin/create_membership.php requires Admin role. Upon form submission, it validates plan_name, duration (integer days), fee (decimal), description, and role (visitor/player/host). It generates plan_id using time-based logic and inserts into MEMBERSHIPPLAN with the role-specific restriction ensuring separate plans for each role.
- The membership purchase in visitor/buy_membership.php and player/handle_membership.php validates plan_id by querying MEMBERSHIPPLAN, checks if user's role matches plan role, generates payment_id, inserts PAYMENT with pay_type membership, amount from plan fee, pay_status pending, and pay_date current date. It then inserts into BUY table linking user_id, payment_id, plan_id, start_date as current date, and end_date calculated by adding duration days to start_date.
- Regular game booking conflict detection in player/room_booking.php validates room_id, booking_date, start_time, end_time, game_name, and num_players. It queries BOOKING for overlapping bookings using the formula

Source Code Repository

<https://github.com/Ratul-w/CSE370-Project/tree/main/Gamezone>

Conclusion

Overall, we gave our best effort to finish this project. There are still some bugs which might require a long time to handle. We are confident that if this project is fully debugged, this project might bring a change in the E-sports industry.

References